

Algorithms for Quad-Double Precision Floating Point Arithmetic*

Yozo Hida
Computer Science Division
University of California
Berkeley, CA 94720
yozo@cs.berkeley.edu

Xiaoye S. Li David H. Bailey
NERSC
Lawrence Berkeley National Laboratory
1 Cyclotron Rd, Berkeley, CA 94720
{xsli, dhbailey}@lbl.gov

Abstract

A quad-double number is an unevaluated sum of four IEEE double precision numbers, capable of representing at least 212 bits of significand. We present the algorithms for various arithmetic operations (including the four basic operations and various algebraic and transcendental operations) on quad-double numbers. The performance of the algorithms, implemented in C++, is also presented.

1. Introduction

Multiprecision computation has a variety of applications, including pure mathematics, physics, cryptography and computational geometry. For example, high precision computations are used in mathematical research to recognize numerical constants and to numerically discover previously unknown mathematical relationships [3]. High precision arithmetic is used in computational physics to explore phenomena that otherwise would be lost in numerical “noise” [4]. It is used in computational geometry to resolve uncertainties in polygons [12].

Many arbitrary precision algorithms and libraries have been developed using only the fixed precision arithmetic. They can be divided into two groups based on the way precision numbers are represented. Some libraries store numbers in a *multiple-digit* format, with a sequence of digits coupled with a single exponent, such as the symbolic computation package Mathematica, Bailey’s MPFUN [2], Brent’s MP [5] and GNU MP [8]. An alternative approach is to store numbers in a *multiple-component* format, where a number is expressed as unevaluated sums of ordinary floating-point words, each with its own significand and exponent. Examples of this format include [7, 11, 12]. The multiple-digit approach can represent a much larger range

of numbers, whereas the multiple-component approach has the advantage in speed.

We note that many applications would get full benefit from using merely a small multiple of (such as twice or quadruple) the working precision, without the need for arbitrary precision. The algorithms for this kind of “fixed” precision can be made significantly faster than those for arbitrary precision. Bailey [1] and Briggs [6] have developed algorithms and software for “double-double” precision, twice the double precision. They used the multiple-component format, where a double-double number is represented as an unevaluated sum of a leading double and a trailing double.

In this paper, we present some algorithms for “quad-double” numbers, in other words, numbers with four times the double precision. We use the multiple-component format to take advantage of speed. A quad-double number is an unevaluated sum of four IEEE doubles. The quad-double number (a_0, a_1, a_2, a_3) represents the exact sum $a = a_0 + a_1 + a_2 + a_3$, where a_0 is the most significant component. We have designed and implemented algorithms for basic arithmetic operations, as well as some algebraic and transcendental functions. We have performed extensive correctness tests and compared the results with arbitrary precision package MPFUN. See [9] for more details about the software.

The rest of the paper is organized as follows. Section 2 describes some basic properties of IEEE floating point arithmetic and the building blocks used in the quad-double algorithms. In Section 3 we present the quad-double algorithms for basic operations, including renormalization, addition, multiplication and division. Section 4 presents the timing results of the C++ implementation on various architectures. Section 5 gives a summary and discusses future work.

2. Preliminaries

In this section, we present some basic properties and algorithms of IEEE floating point arithmetic used in quad-double arithmetic. These results are based on Dekker [7],

*This research was supported by the Director, Office of Science, Division of Mathematical, Information, and Computational Sciences of the U.S. Department of Energy under contract number DE-AC03-76SF00098.

Knuth [10], Priest [11], Shewchuk [12], and others.

All basic arithmetics are assumed to be performed in IEEE double format, with round-to-even rounding on ties. For any binary operator $\cdot \in \{+, -, \times, /\}$, we use $\text{fl}(a \cdot b) = a \odot b$ to denote the floating point result of $a \cdot b$, and define $\text{err}(a \cdot b)$ as $a \cdot b = \text{fl}(a \cdot b) + \text{err}(a \cdot b)$. Throughout this paper, $\varepsilon = 2^{-53}$ is the machine epsilon for IEEE double precision numbers, and $\varepsilon_{\text{qd}} = 2^{-211}$ is the precision one expects for quad-double numbers.

Lemma 1. [12, p. 310] *Let a and b be two p -bit floating point numbers such that $|a| \geq |b|$. Then $|\text{err}(a + b)| \leq |b| \leq |a|$.*

Lemma 2. [12, p. 311] *Let a and b be two p -bit floating point numbers. Then $\text{err}(a + b) = (a + b) - \text{fl}(a + b)$ is representable as a p -bit floating point number.*

Algorithm 3. [12, p. 312] The following algorithm computes $s = \text{fl}(a + b)$ and $e = \text{err}(a + b)$, assuming $|a| \geq |b|$.

QUICK-TWO-SUM(a, b)
 1. $s \leftarrow a \oplus b$
 2. $e \leftarrow b \ominus (s \ominus a)$
 3. **return** (s, e)

Algorithm 4. [12, p. 314] The following algorithm computes $s = \text{fl}(a + b)$ and $e = \text{err}(a + b)$. This algorithm uses three more floating point operations instead of a branch.

TWO-SUM(a, b)
 1. $s \leftarrow a \oplus b$
 2. $v \leftarrow s \ominus a$
 3. $e \leftarrow (a \ominus (s \ominus v)) \oplus (b \ominus v)$
 4. **return** (s, e)

Algorithm 5. [12, p. 325] The following algorithm splits a 53-bit IEEE double precision floating point number into a_{hi} and a_{lo} , each with 26 bits of significand, such that $a = a_{\text{hi}} + a_{\text{lo}}$. a_{hi} will contain the first 26 bits, while a_{lo} will contain the lower 26 bits.

SPLIT(a)
 1. $t \leftarrow (2^{27} + 1) \otimes a$
 2. $a_{\text{hi}} \leftarrow t \ominus (t \ominus a)$
 3. $a_{\text{lo}} \leftarrow a \ominus a_{\text{hi}}$
 4. **return** ($a_{\text{hi}}, a_{\text{lo}}$)

Algorithm 6. [12, p. 326] The following algorithm computes $p = \text{fl}(a \times b)$ and $e = \text{err}(a \times b)$.

TWO-PROD(a, b)
 1. $p \leftarrow a \otimes b$
 2. $(a_{\text{hi}}, a_{\text{lo}}) \leftarrow \text{SPLIT}(a)$
 3. $(b_{\text{hi}}, b_{\text{lo}}) \leftarrow \text{SPLIT}(b)$
 4. $e \leftarrow ((a_{\text{hi}} \otimes b_{\text{hi}} \ominus p) \oplus a_{\text{hi}} \otimes b_{\text{lo}} \oplus a_{\text{lo}} \otimes b_{\text{hi}}) \oplus a_{\text{lo}} \otimes b_{\text{lo}}$
 5. **return** (p, e)

Some machines have a fused multiply-add instruction (FMA) that can evaluate expression such as $a \times b \pm c$ with

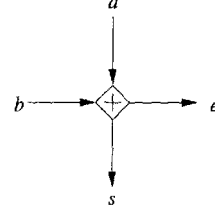


Figure 1. QUICK-TWO-SUM (3 FLOPS)

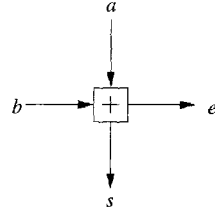


Figure 2. TWO-SUM (6 FLOPS)

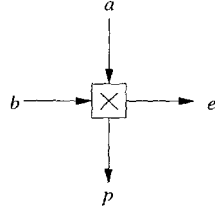


Figure 3. TWO-PROD (17 FLOPS) and TWO-PROD-FMA (3 FLOPS)

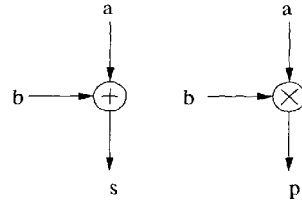


Figure 4. Normal IEEE double precision sum and product

a single rounding error. We can take advantage of this instruction to compute the exact product of two floating point numbers much faster. These machines include IBM Power series (including the PowerPC), on which this simplification is tested.

Algorithm 7. The following algorithm computes $p = \text{fl}(a \times b)$ and $e = \text{err}(a \times b)$ on a machine with a FMA instruction. Note that some compilers emit FMA instructions for $a \times b + c$ but not for $a \times b - c$; in this case, some sign adjustments must be made.

```
TWO-PROD-FMA( $a, b$ )
1.  $p \leftarrow a \otimes b$ 
2.  $e \leftarrow \text{fl}(a \times b - p)$ 
3. return ( $p, e$ )
```

The algorithms presented are the basic building blocks of quad-double arithmetic, and are represented in Figures 1, 2, and 3. In the caption of each figure, we give the count of normal floating point operations (FLOPS) associated with each building block. Symbols for normal double precision sum and product are in Figure 4.

3. Basic Operations

In this section, we present our algorithms for basic arithmetic operations. The other algorithms for algebraic and transcendental functions are described in [9].

Since each component of a quad-double number is a IEEE double, the exponent range is limited to that of a double precision number. This implies that the arithmetic operations on quad-double numbers near underflow may result in a loss of precision.

3.1. Renormalization

A quad-double number is an unevaluated sum of four IEEE double numbers. The quad-double number (a_0, a_1, a_2, a_3) represents the exact sum $a = a_0 + a_1 + a_2 + a_3$. Note that for any given representable number x , there can be many representations as an unevaluated sum of four doubles. Hence we require that the quadruple (a_0, a_1, a_2, a_3) to satisfy

$$|a_{i+1}| \leq \frac{1}{2} \text{ulp}(a_i), \text{ for } i = 0, 1, 2$$

with equality occurring only if $a_i = 0$ or the last bit of a_i is 0 (that is, round-to-even is used in case of ties). Note that the first double a_0 is a double-precision approximation to the quad-double number a , accurate to almost half an ulp.

Lemma 8. *For any quad-double number $a = (a_0, a_1, a_2, a_3)$, the normalized representation is unique.*

Most of the algorithms described here produce an expansion that is not of canonical form — often having overlapping bits. Therefore, a five-term expansion is produced, and then renormalized to four components. Note, two floating point numbers x and y are overlapping if the least significant bit of x is smaller than the most significant bit of y .

Algorithm 9. This renormalization procedure is a variant of Priest's renormalization method [11, p. 116]. The input is a five-term expansion with limited overlapping bits, with a_0 being the most significant component.

```
RENORMALIZE( $a_0, a_1, a_2, a_3, a_4$ )
1.  $(s, t_4) \leftarrow \text{QUICK-TWO-SUM}(a_3, a_4)$ 
2.  $(s, t_3) \leftarrow \text{QUICK-TWO-SUM}(a_2, s)$ 
3.  $(s, t_2) \leftarrow \text{QUICK-TWO-SUM}(a_1, s)$ 
4.  $(t_0, t_1) \leftarrow \text{QUICK-TWO-SUM}(a_0, s)$ 
5.  $s \leftarrow t_0$ 
6.  $k \leftarrow 0$ 
7. for1  $i \leftarrow 1, 2, 3, 4$ 
8.    $(s, e) \leftarrow \text{QUICK-TWO-SUM}(s, t_i)$ 
9.   if  $e \neq 0$ 
10.     $b_k \leftarrow s$ 
11.     $s \leftarrow e$ 
12.     $k \leftarrow k + 1$ 
13.  end if
14. end for
15. return ( $b_0, b_1, b_2, b_3$ )
```

Necessary conditions for this renormalization algorithm to work correctly are, unfortunately, not known. Priest proves that if the input expansion does not overlap by more than 51 bits, then the algorithm works correctly. However, this condition is by no means necessary; that the renormalization algorithm (Algorithm 9) works on all the expansions produced by the algorithms below remains to be shown.

3.2. Addition

Quad-Double + Double. The addition of a double precision number to a quad-double number is similar to Shewchuk's GROW-EXPANSION [12, p. 316], but the double precision number b is added to a quad-double number a from most significant component first (rather than from least significant). This produces a five-term expansion which is the exact result, which is then renormalized. See Figure 5.

Since the exact result is computed, then normalized to four components, this addition is accurate to at least the first 212 bits of the result.

Quad-Double + Quad-Double. We have implemented two algorithms for addition. The first one is faster, but only satisfies the weaker (Cray-style) error bound $a \oplus b =$

¹In the implementation, this loop is unrolled to several **if** statements.

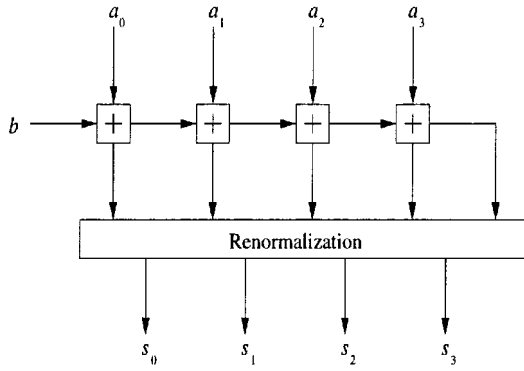


Figure 5. Quad-Double + Double

$(1 + \delta_1)a + (1 + \delta_2)b$ where the magnitude of δ_1 and δ_2 is bounded by $\varepsilon_{qd} = 2^{-211}$.

Figure 6 best describes the first addition algorithm of two quad-double numbers. In the diagram, there are three large boxes with three inputs to them. These are various THREE-SUM boxes, and their internals are shown in Figure 7.

Now for a few more lemmas.

Lemma 10. *Let a and b be two double precision floating point numbers. Let $M = \max(|a|, |b|)$. Then $|\text{fl}(a + b)| \leq 2M$, and consequently, $|\text{err}(a + b)| \leq \frac{1}{2} \text{ulp}(2M) \leq 2\varepsilon M$.*

Lemma 11. *Let x, y , and z be inputs to THREE-SUM. Let u, v, w, r_0, r_1 , and r_2 be as indicated in Figure 7. Let $M = \max(|x|, |y|, |z|)$. Then $|r_0| \leq 4M$, $|r_1| \leq 8\varepsilon M$, and $|r_2| \leq 8\varepsilon^2 M$.*

Proof. This follows from applying Lemma 10 to each of the three TWO-SUM boxes. First TWO-SUM gives $|u| \leq 2M$ and $|v| \leq 2\varepsilon M$. Next TWO-SUM (adding u and z) gives $|r_0| \leq 4M$ and $|w| \leq 4\varepsilon M$. Finally, the last TWO-SUM gives the desired result. $\square$

Note that the two other THREE-SUMS shown are simplification of the first THREE-SUM, where it only computes one or two components, instead of three; thus the same bounds apply.

The above bound is not at all tight; $|r_0|$ is bounded closer to $3M$ (or even $|x| + |y| + |z|$), and this makes the bounds for r_1 and r_2 correspondingly smaller. However, this suffices for the following lemma.

Lemma 12. *The five-term expansion before the renormalization step in the quad-double addition algorithm shown in Figure 6 errs from the true result by less than $\varepsilon_{qd}M$, where $M = \max(|a|, |b|)$.*

Proof. This can be shown by judiciously applying Lemmas 10 and 11 to all the TWO-SUMS and THREE-SUMS in Figure 6. See [9] for a detailed proof. $\square$

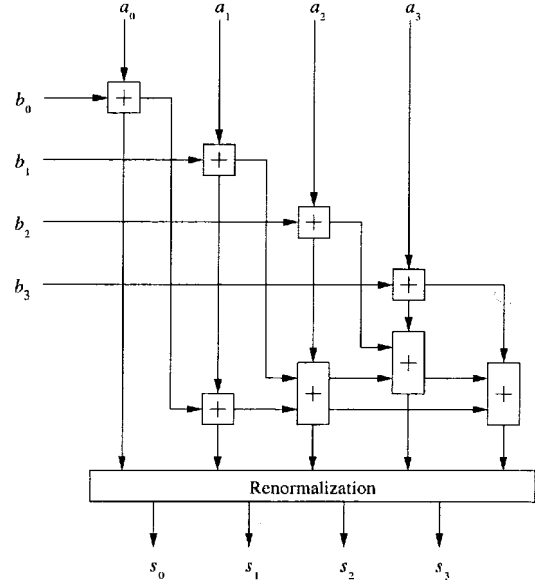


Figure 6. Quad-Double + Quad-Double

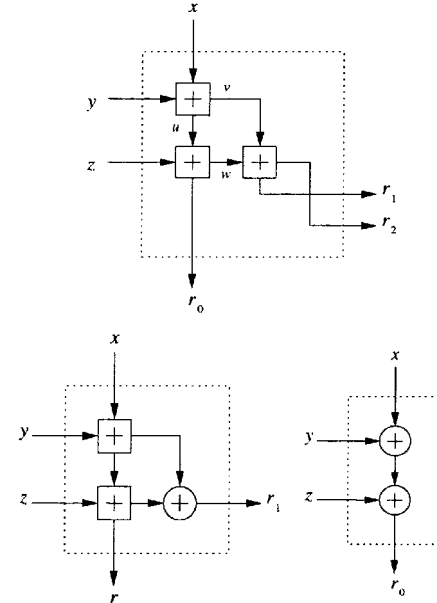


Figure 7. THREE-SUMS

Assuming that the renormalization step works (this remains to be proven), we can then obtain the error bound

$$\text{fl}(a + b) = (1 + \delta_1)a + (1 + \delta_2)b, \text{ with } |\delta_1|, |\delta_2| \leq \varepsilon_{\text{qd}}.$$

Note that the above algorithm for addition is particularly suited to modern processors with instruction level parallelism, since the first four TWO-SUMs can be evaluated in parallel. Lack of branches before the renormalization step also helps to keep the pipelines full. The above algorithm does not satisfy the IEEE-style error bound

$$\text{fl}(a + b) = (1 + \delta)(a + b), \text{ with } |\delta| \leq 2\varepsilon_{\text{qd}} \text{ or so.}$$

To see this, let $a = (u, v, w, x)$ and $b = (-u, -v, y, z)$, where none of w, x, y, z overlaps and $|w| > |x| > |y| > |z|$. Then the above algorithm produces $c = (w, x, y, 0)$ instead of $c = (w, x, y, z)$ required by the stricter bound.

The second algorithm (14), due to J. Shewchuk and S. Boldo, computes the first four components of the *result* correctly. Thus it satisfies stricter error bound

$$\text{fl}(a + b) = (1 + \delta)(a + b), \text{ with } |\delta| \leq 2\varepsilon_{\text{qd}} \text{ or so.}$$

However, it has a corresponding speed penalty; it runs significantly slower (a factor of 2–3.5 slower).

The algorithm is similar to Shewchuk's FAST-EXPANSION-SUM [12, p. 320], where it merge-sorts the two expansions. To prevent components with only a few significant bits to be produced, a double-length accumulator is used so that a component is output only if the inputs gets small enough to not affect it.

Algorithm 13. Assuming that u, v is a two-term expansion, the following algorithm computes the sum $(u, v) + x$, and outputs the significant component s if the remaining components contain more than one double worth of significand. u and v are modified to represent the other two components in the sum.

```

DOUBLE-ACCUMULATE( $u, v, x$ )
1.  $(s, v) \leftarrow \text{TWO-SUM}(v, x)$ 
2.  $(s, u) \leftarrow \text{TWO-SUM}(u, s)$ 
3. if  $u = 0$ 
4.    $u \leftarrow s; \quad s \leftarrow 0$ 
5. end if
6. if  $v = 0$ 
7.    $v \leftarrow u; \quad u \leftarrow s; \quad s \leftarrow 0$ 
8. end if
9. return  $(s, u, v)$ 

```

The accurate addition scheme is as follows.

Algorithm 14. This algorithm computes the sum of two quad-double numbers $a = (a_0, a_1, a_2, a_3)$ and $b = (b_0, b_1, b_2, b_3)$. It merge-sorts the eight doubles, and performs DOUBLE-ACCUMULATE until four components are obtained.

QD-ADD-ACCURATE(a, b)

```

1.  $(x_0, x_1, \dots, x_7) \leftarrow$ 
   MERGE-SORT( $a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3$ )
2.  $u \leftarrow 0; v \leftarrow 0; k \leftarrow 0; i \leftarrow 0$ 
3. while  $k < 4$  and  $i < 8$  do
4.    $(s, u, v) \leftarrow \text{DOUBLE-ACCUMULATE}(u, v, x_i)$ 
5.   if  $s \neq 0$ 
6.      $c_k \leftarrow s; \quad k \leftarrow k + 1$ 
7.   end if
8.    $i \leftarrow i + 1$ 
9. end while
10. if  $k < 2$  then  $c_{k+1} \leftarrow v$ 
11. if  $k < 3$  then  $c_k \leftarrow u$ 
12. return RENORMALIZE( $c_0, c_1, c_2, c_3$ )

```

3.3 Subtraction

Subtraction $a - b$ is implemented as the addition $a + (-b)$, so it has the same algorithm and properties as that of addition. To negate a quad-double number, we can negate each component. On a modern C++ compiler with inlining, the overhead is noticeable but not prohibitive (say 5% or so).

3.4 Multiplication

Multiplication is done in a straightforward manner, multiplying term by term and accumulating. Unlike addition, there are no possibilities of massive cancellation in multiplication, so the following algorithms satisfy the IEEE style error bound $a \otimes b = (1 + \delta)(a \times b)$, with $|\delta| \leq \varepsilon_{\text{qd}}$.

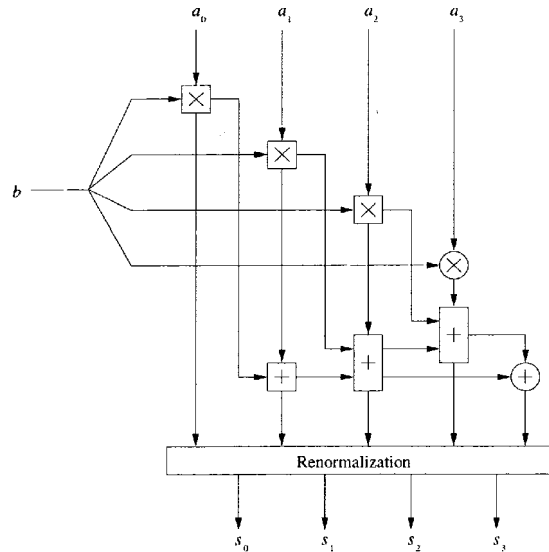


Figure 8. Quad-Double $\times$ Double

Quad-Double $\times$ Double. Let $a = (a_0, a_1, a_2, a_3)$ be a quad-double number, and let b be a double precision number. Then the product is the sum of four terms, $a_0b + a_1b + a_2b + a_3b$. Note that $|a_3| \leq \varepsilon^3|a_0|$, so $|a_3b| \leq \varepsilon^3|a_0b|$, and thus only the first 53 bits of the product a_3b need to be computed. The first three terms are computed exactly using TWO-PROD (or TWO-PROD-FMA). All the terms are then accumulated in a similar fashion as addition. See Figure 8.

Quad-Double $\times$ Quad-Double. Multiplication of two quad-double numbers becomes a bit complicated, but nevertheless follows the same idea. Let $a = (a_0, a_1, a_2, a_3)$ and $b = (b_0, b_1, b_2, b_3)$ be two quad-double numbers. Assume (without loss of generality) that a and b are order 1. After multiplication, we need to accumulate 13 terms of order $O(\varepsilon^4)$ or lower.

$$\begin{aligned}
 a \times b &\approx a_0b_0 && O(1) \text{ term} \\
 &+ a_0b_1 + a_1b_0 && O(\varepsilon) \text{ terms} \\
 &+ a_0b_2 + a_1b_1 + a_2b_0 && O(\varepsilon^2) \text{ terms} \\
 &+ a_0b_3 + a_1b_2 + a_2b_1 + a_3b_0 && O(\varepsilon^3) \text{ terms} \\
 &+ a_1b_3 + a_2b_2 + a_3b_1 && O(\varepsilon^4) \text{ terms}
 \end{aligned}$$

Note that smaller order terms (such as a_2b_3 , which is $O(\varepsilon^5)$) are not even computed, since they are not needed to get the first 212 bits. The $O(\varepsilon^4)$ terms are computed using normal double precision arithmetic, as only their first few bits are needed.

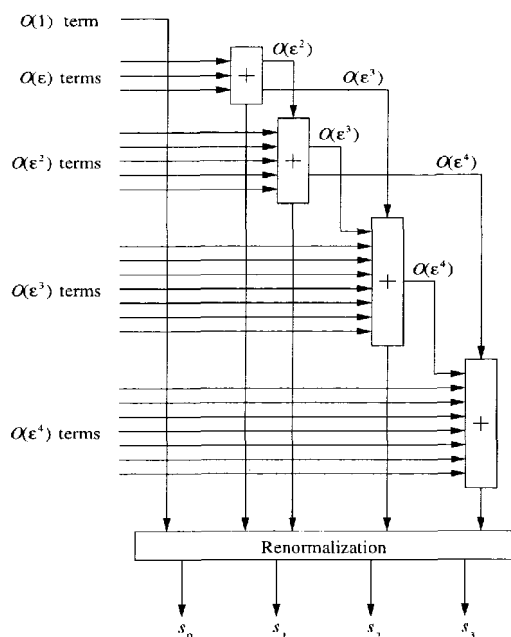


Figure 9. Quad-Double $\times$ Quad-Double accumulation phase

For $i + j \leq 3$, let $(p_{ij}, q_{ij}) = \text{TWO-PROD}(a_i, b_j)$. Then $p_{ij} = O(\varepsilon^{i+j})$ and $q_{ij} = O(\varepsilon^{i+j+1})$. Now there are one term (p_{00}) of order $O(1)$, three (p_{01}, p_{10}, q_{00}) of order $O(\varepsilon)$, five ($p_{02}, p_{11}, p_{20}, q_{01}, q_{10}$) of order $O(\varepsilon^2)$, seven of order $O(\varepsilon^3)$, and seven of order $O(\varepsilon^4)$. Now we can start accumulating all the terms by their order, starting with $O(\varepsilon)$ terms (see Figure 9).

In the diagram, there are four different summation boxes. The first (topmost) one is THREE-SUM, same as the one in addition. The next three are, respectively, SIX-THREE-SUM (sums six doubles and outputs the first three components), NINE-TWO-SUM (sums nine doubles and outputs the first two components), and NINE-ONE-SUM (just adds nine doubles using normal arithmetic).

SIX-THREE-SUM computes the sum of six doubles to three double worth of accuracy (i.e., to relative error of $O(\varepsilon^3)$). This is done by dividing the inputs into two groups of three, and performing THREE-SUM on each group. Then the two sums are added together, in a manner similar to quad-double addition. See Figure 10.

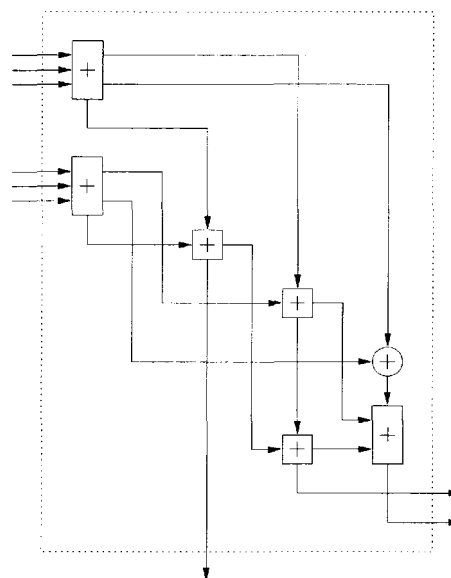


Figure 10. SIX-THREE-SUM

NINE-TWO-SUM computes the sum of nine doubles to double-double accuracy. This is done by pairing the inputs to create four double-double numbers and a single double precision number, and performing addition of two double-double numbers recursively until one arrives at a double-double output. The double-double addition (the large square box in the diagram) is the same as David Bailey's algorithm [1]. See Figure 11.

If one wishes to trade a few bits of accuracy for speed, we don't even need to compute the $O(\varepsilon^4)$ terms; they can

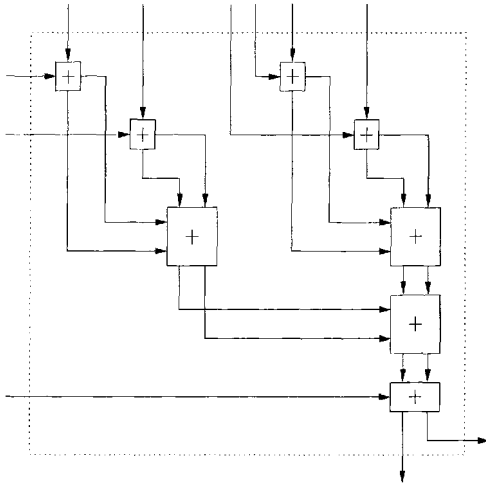


Figure 11. NINE-TWO-SUM

affect the first 212 bits only by carries during accumulation. In this case, we can compute the $O(\varepsilon^3)$ terms using normal double precision arithmetic, thereby speeding up multiplication considerably.

Squaring a quad-double number can be done significantly faster since the number of terms that needs to be accumulated can be reduced due to symmetry.

3.5. Division

Division is done by the familiar long division algorithm. Let $a = (a_0, a_1, a_2, a_3)$ and $b = (b_0, b_1, b_2, b_3)$ be quad-double numbers. We can first compute an approximate quotient $q_0 = a_0/b_0$. We then compute the remainder $r = a - q_0 \times b$, and compute the correction term $q_1 = r_0/b_0$. We can continue this process to obtain five terms, q_0, q_1, q_2, q_3 , and q_4 . (only four are needed if we can afford to sacrifice a few bits of accuracy).

Note that at each step, full quad-double multiplication and subtraction must be done since most of the bits will be canceled when computing q_3 and q_4 . The five-term (or four-term) expansion is then renormalized to obtain the quad-double quotient.

4. Performance and Application

We have implemented the above quad-double algorithms in ANSI C++. In this section, we demonstrate the performance of the quad-double library.

Table 4 shows the measurements of various kernel operations on quad-double numbers on a variety of machines. The machines, operating systems, compilers and optimizations we used are listed below:

Operation	Pentium II	UltraSparc	PowerPC	Power3
<i>Quad-double</i>				
add	0.583	0.580	0.868	0.710
accurate add	1.280	2.464	2.468	1.551
mul	1.965	1.153	1.744	1.131
sloppy mul	1.016	0.860	1.177	0.875
div	5.267	6.440	8.210	6.699
sloppy div	4.080	4.163	6.200	4.979
sqrt	23.646	15.003	21.415	16.174
<i>MPFUN</i>				
add	5.729	5.362	—	4.651
mul	7.624	7.630	—	5.837
div	10.102	10.164	—	9.180

Table 1. Performance of some Quad-Double algorithms on several machines. All measurements are in microseconds. We include the performance of MPFUN [2] as a comparison. Note, we do not have the MPFUN measurements on the PowerPC, because we do not have a Fortran-90 compiler.

- Intel Pentium II, 400 MHz, Linux 2.2.16, g++ 2.95.2 compiler, with -O3 -funroll-loops -finline-functions -mcpu=i686 -march=i686
- Sun UltraSparc 333 MHz, SunOS 5.7, Sun CC 5.0 compiler, with -xO5 -native
- PowerPC 750 (Apple G3), 266 MHz, Linux 2.2.15, g++ 2.95.2 compiler, with -O3 -funroll-loops -finline-functions
- IBM RS/6000 Power3, 200 MHz, AIX 3.4, IBM xlc compiler, with -O3 -qarch=pwr3 -qtune=pwr3 -qstrict

Our quad-double library was successfully used in a parallel vortex roll-up simulation [4], which uses various transcendental functions as well as basic operations. On the NERSC IBM SP, using 256 Power3 processors, the quad-double version runs about four times as fast as the multi-precision (MPFUN) version, and delivers almost identical results.

5. Summary and Future Work

We presented the algorithms and performance of various operations on quad-double precision numbers. The algorithms assume that a quad-double number is represented as an unevaluated sum of four IEEE double precision numbers. Note that the simpler approach of reusing double-double arithmetic recursively to perform arithmetic on quad-double numbers does not work, as the double-double algorithms require IEEE behaviour on the base arithmetic type. Double-double numbers are not analogue of a IEEE-style "quadru-

ple” numbers, as the two differs significantly in their behaviour and the numbers they can represent. As an example, $1.0 + 2^{-1000}$ is not representable in IEEE style quadruple precision floating point number but is a perfectly valid double-double number ($1.0, 2^{-1000}$).

Another approach is to use generic algorithms that operate on two multicomponent numbers and use them recursively to define quad-double arithmetic. For example, we can take Shewchuk’s multiprecision library and modify it to throw away unnecessary computations. This approach, however, does not allow for special optimizations that assumes the number of component is limited to 4. Our algorithms perform significantly better primarily for this reason, at the cost of generality and unproven correctness. However for numbers with larger number of components, the benefit of special optimizations diminish, and adaptation of multiprecision arithmetic is probably a better approach.

Our algorithms are implemented in a C++ library, taking full advantage of operator/function overloading and user-defined data structures. Our quad-double library also contains miscellaneous supporting routines, including input/output, comparisons, and random number generation. The complete package and the details about its usage, testing and C/Fortran interfaces can be found in [9].

We have yet to provide the full correctness proof for the basic routines. The correctness of these routines rely on the fact that renormalization step works; Priest proves that it does work if the input does not overlap by 51 bits and no three components overlap at a single bit. Whether such overlap can occur in any of these algorithm needs to be proved.

There are improvements due in the remainder operator, which computes $a - \text{round}(a/b) \times b$, given quad-double numbers a and b . Currently, the library does the naïve method of just divide, round, multiply, and subtract. This leads to loss of accuracy when a is large compared to b . Since this routine is used in argument reduction for exponentials, logarithms and trigonometrics, a fix is needed.

A natural extension of this work is to extend the precision beyond quad-double. Algorithms for quad-double additions and multiplication can be extended to higher precisions, however, with more components, asymptotically faster algorithm by S. Boldo and J. Shewchuk may be preferable (i.e. Algorithm 14). One limitation with these higher precision expansions is the limited exponent range – same as that of double. Hence the maximum precision is about 2000 bits (39 components), and this occurs only if the first component is near overflow and the last near underflow.

Acknowledgments

We thank Jonathan Shewchuk, Sylvie Boldo, and James Demmel for constructive discussions. The accurate version

of addition algorithm is due to S. Boldo and J. Shewchuk. Problems with remainder were pointed out by J. Demmel.

References

- [1] D. H. Bailey. A fortran-90 double-double library. Available at <http://www.nersc.gov/~dhbailey/mpdist/mpdist.html>.
- [2] D. H. Bailey. A fortran-90 based multiprecision system. *ACM Transactions on Mathematical Software*, 21(4):379–387, 1995. Available at <http://www.nersc.gov/~dhbailey/mpdist/mpdist.html>.
- [3] D. H. Bailey. Integer relation detection. *Computing in Science and Engineering*, pages 24–28, Jan-Feb 2000.
- [4] D. H. Bailey, R. Krasny, and R. Pelz. Multiple precision, multiple processor vortex sheet roll-up computation. *Proc. Sixth SIAM Conference on Parallel Processing for Scientific Computing*, pages 52–56, 1993.
- [5] R. Brent. A Fortran multiple precision arithmetic package. *ACM Trans. Math. Soft.*, 4:57–70, 1978.
- [6] K. Briggs. Doubledouble floating point arithmetic. <http://www-epidem.plantsci.cam.ac.uk/~kbriggs/doubledouble.html>, 1998.
- [7] T. Dekker. A floating-point technique for extending the available precision. *Numerische Mathematik*, 18:224–242, 1971.
- [8] GMP. <http://www.swox.com/gmp/>.
- [9] Y. Hida, X. S. Li, and D. H. Bailey. Quad-double arithmetic: Algorithms, implementation, and application. Technical Report LBNL-46996, Lawrence Berkeley National Laboratory, Berkeley, CA 94720, October 2000. Available at <http://www.nersc.gov/~dhbailey/mpdist/mpdist.html>.
- [10] D. E. Knuth. *The Art of Computer Programming: Seminumerical Algorithms*, volume 2. Addison Wesley, Reading, Massachusetts, 1981.
- [11] D. M. Priest. *On Properties of Floating Point Arithmetics: Numerical Stability and the Cost of Accurate Computations*. PhD thesis, University of California, Berkeley, November 1992. Available by anonymous FTP at <ftp.icsi.berkeley.edu/pub/theory/priest-thesis.ps.Z>.
- [12] J. R. Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18(3):305–363, 1997.